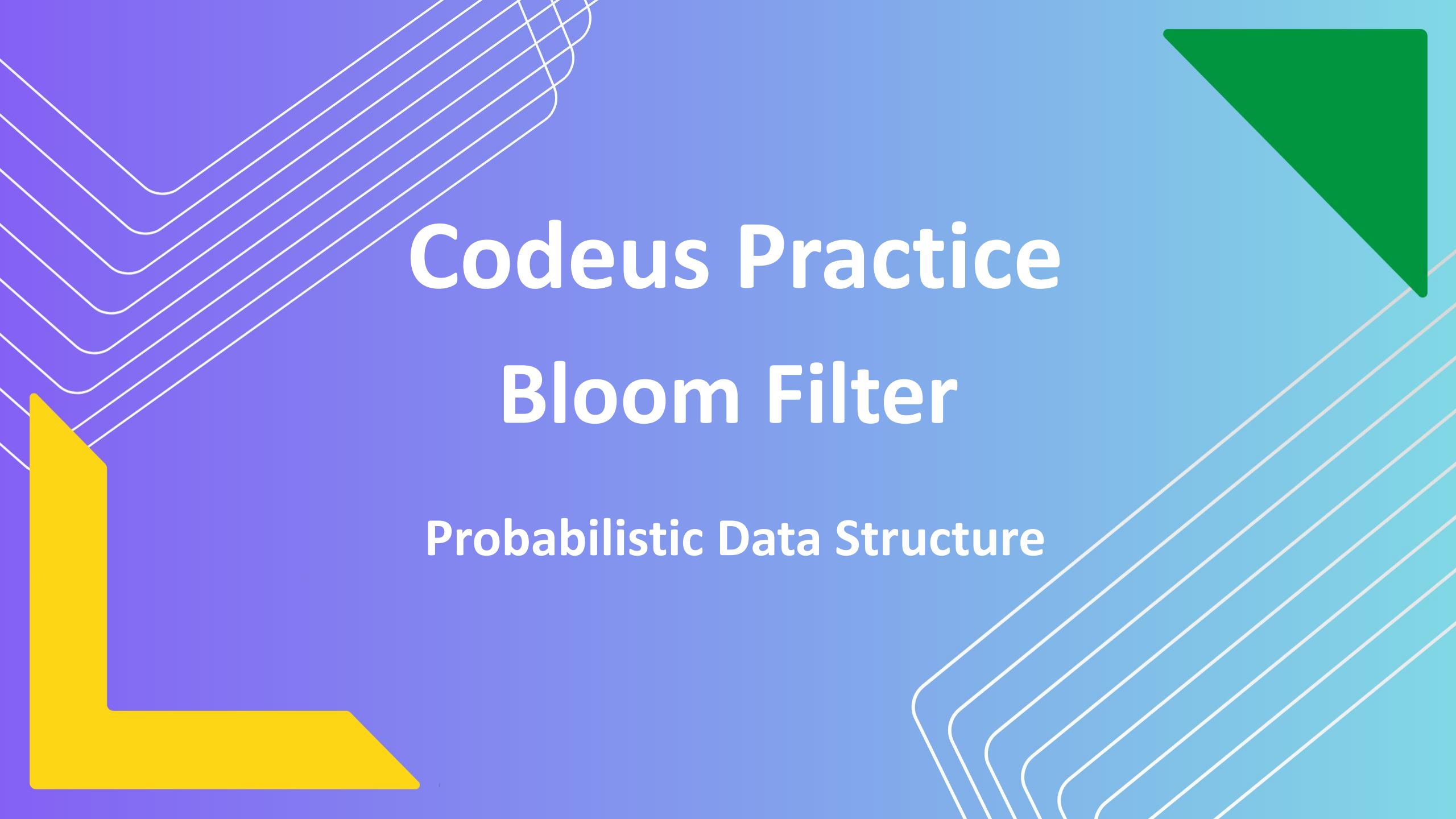




Codeus Practice

Bloom Filter

Probabilistic Data Structure



About me

- Name: Serhii Horbachov
- Software engineer @EPAM



Bloom Filter

Probabilistic data structure, conceived by Burton Howard Bloom in 1970

Checks if element might be present or definitely not present in a set.

Guarantees no false negatives (if it says “not present,” it’s true) but

Allows false positives (may say “present” when it’s not).



Problem

Avoiding expensive lookups

Reducing memory usage

Speeding up membership tests



Application of Bloom Filters



Databases & Storage Systems

Used to minimize disk access and speed up lookups.



Networking & Web Systems

Helps reduce latency and unnecessary network calls (CDNs, web caches, peer-to-peer systems).



User Experience & Recommendations

Improves relevance while saving compute resources.

- Prevents showing users items they've already seen or interacted with.
- Helps track viewed content in recommender systems (e.g., news feeds, e-commerce).



Application of Bloom Filters



Big Data & Stream Processing

Ideal for large-scale or continuous data pipelines.

- Deduplicate events or messages in streaming systems (Kafka, Spark Streaming).
- Prevent reprocessing the same data in ETL jobs.
- Filter data efficiently before it reaches expensive downstream operations.



Security & Privacy

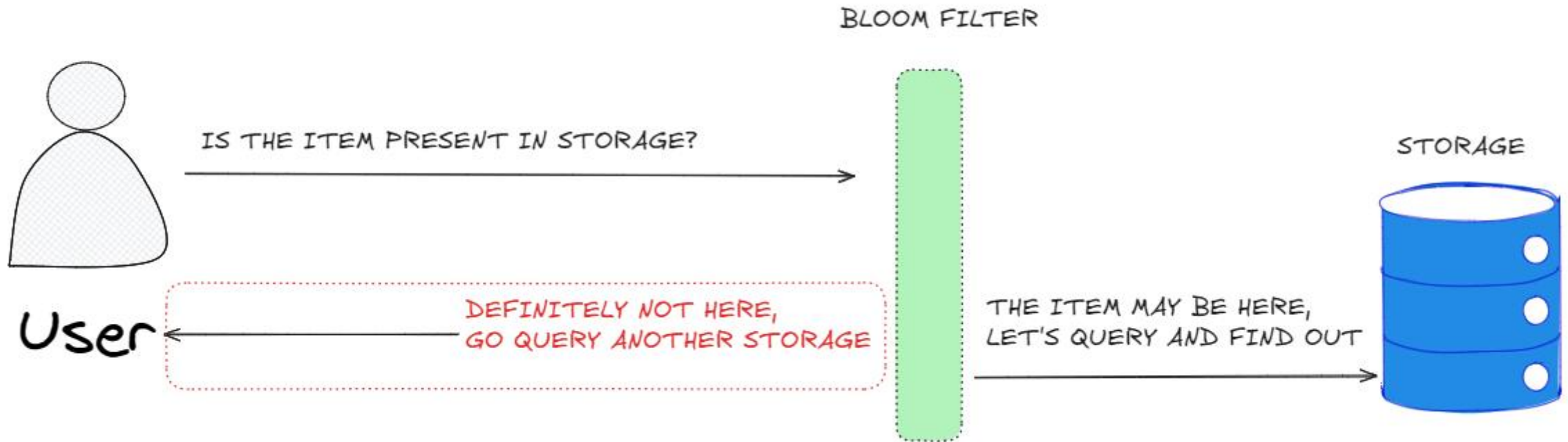
Used to block or flag suspicious data patterns quickly.

- Detect blacklisted IPs, URLs, or emails without large in-memory tables.
- Check password breach lists (e.g., Have I Been Pwned uses Bloom Filters).
- Identify known malicious signatures in firewalls or intrusion detection systems.



How does Bloom Filter work?

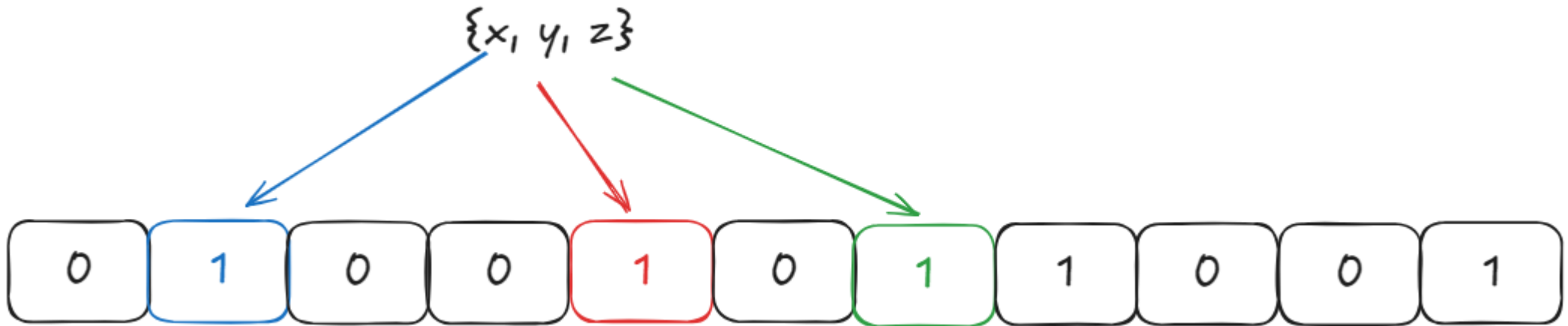
Bloom Filter



Bloom Filter

Consists of a block of memory that's "m" bits long with some number "k" of hash functions.
x, y, z – hash functions.

Hash the item using each hash function and set a bit to the corresponding memory location



Bloom Filter – Membership check

1. Hash the Item Multiple Times

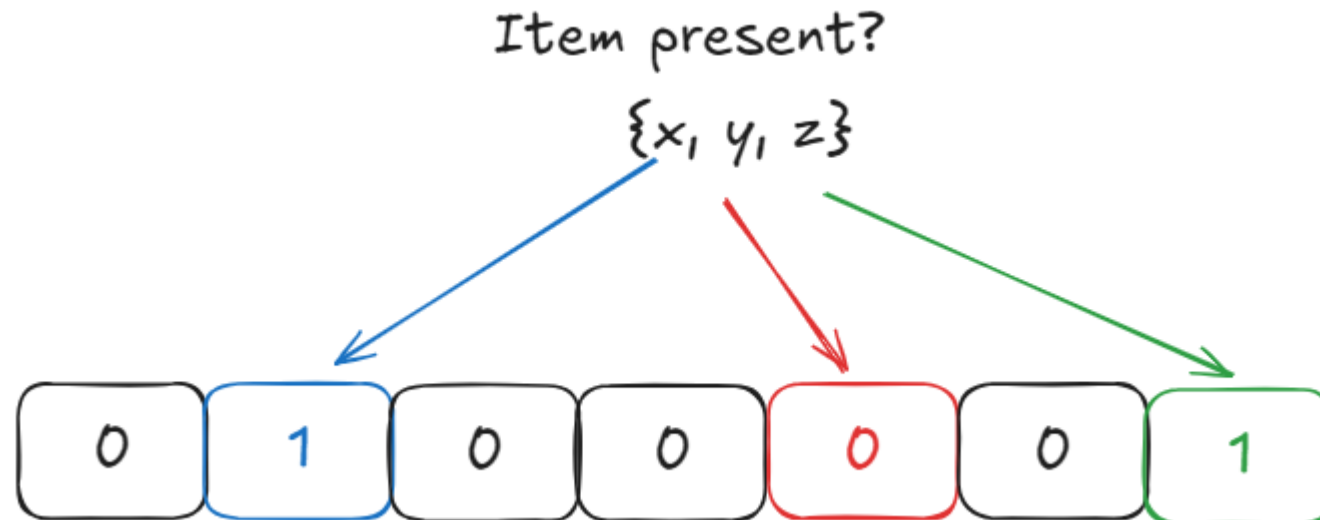
Use the same set of hash functions that were used when adding items to the filter.

2. Check the Corresponding Bits:

For each hash, calculate the index in the bit array and check if the bit at that index is set to **true**.

3. Interpret the Result:

- If all bits are true: The item **might be present** (could be a false positive).
- If any bit is false: The item is **definitely not present**.



Bloom Filter Time Complexity

Time Complexity

Operation	Complexity	Explanation
Insert (add)	$O(k)$	Compute k hash functions and set k bits.
Query (contains)	$O(k)$	Compute k hashes and check k bits.
Delete	 Not supported in standard Bloom Filter (only Counting Bloom Filter allows it).	



Bloom Filter Space / Memory Requirements

Memory depends on:

$$m = - \frac{n \cdot \ln(p)}{(\ln 2)^2}$$

where

- m = number of bits in the filter
- n = number of expected elements
- p = desired false positive probability

Example:

For $n = 1,000,000$ elements and $p = 1\%$ (0.01)

→ $m \approx 9.6$ million bits ≈ 1.2 MB of memory

Extremely space-efficient — about **10 bits per element** achieves a ~1% false positive rate.



Bloom Filter Optimal Number of Hash Functions

$$k = \frac{m}{n} \ln 2$$

Where

m = number of bits in the filter

n = number of expected elements

For 1% false positives $\rightarrow k \approx 7$

Fewer hashes = faster, more memory usage

More hashes = slower, fewer false positives



Bloom Filter False Positive Rate

$$p = \left(1 - e^{-kn/m}\right)^k$$

where

k = number of hash functions

m = number of bits in the filter

n = number of expected elements

Increases as more elements are added (since more bits get set to 1).

No false negatives — only false positives.



Bloom Filter Performance Characteristics

Insert and query: constant time.

Parallelizable: all hashes and bit operations can be computed independently.

Cache-friendly: bit array fits easily into memory; no pointers or object overhead.



Bloom Filter

How to select hash function?

In Bloom filters, hash functions should ideally have two main properties:

Uniform Distribution:

The hash function should distribute the output (hash values) uniformly across the bit array. This ensures that each bit in the array is equally likely to be set, minimizing the chances of collisions and thereby reducing false positives.

Independence:

Multiple hash functions used in a Bloom filter should be independent of each other. This means that the outcome of one hash function should not influence the outcome of another.



Bloom Filter Pros/Cons

 Benefit	 Disadvantages (Cons)
Space-efficient	False positives
Fast lookups	No deletions (standard version)
No need to store data	Performance depends on tuning Requires choosing good hash functions and correct size for expected data.
Scales to huge datasets	False positive rate grows with usage



Counting Bloom Filter

How it works:

Instead of a bit array, it uses an array of counters (e.g., `int[]`).

Add:

Increment the counters at each hash position.

Remove:

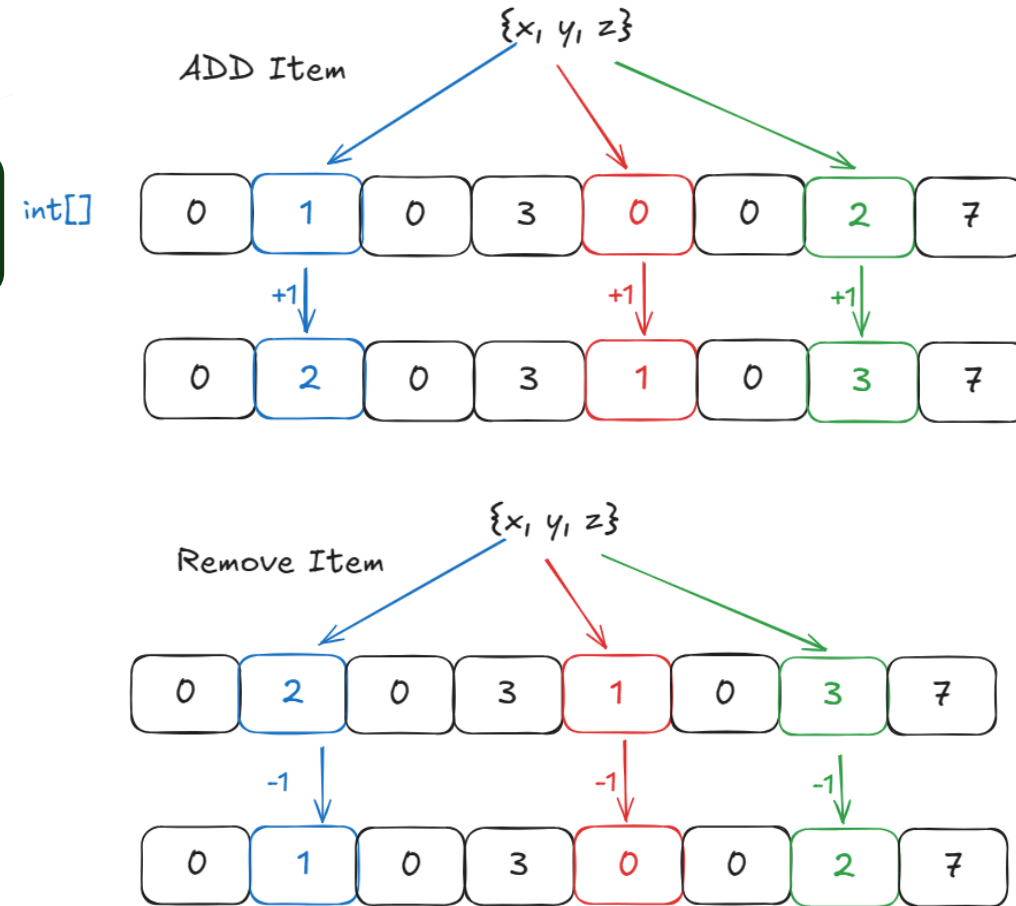
Decrement the counters at each hash position.

Membership check:

All counters at the hash positions must be greater than zero.

Use case:

When you need to add and remove items, such as in dynamic sets or caches.



Counting Bloom Filter Limitations



Potential for false negatives!

Approach	Effectiveness	Notes
Check mightContain before remove	Low	Avoids unnecessary decrements, but not collisions
Decrement counters always	Standard	Accepts risk of false negatives
Use larger filter / better hashes	High	Reduces collision risk
Use alternative data structure	Very High	No false positives/negatives, more memory



Counting Bloom Filter Summary



Potential for false negatives!

Operation	Standard Bloom Filter	Counting Bloom Filter	Hash Set
Add	Yes	Yes	Yes
Remove	No	Yes (with risk)	Yes
False Positives	Yes	Yes	No
False Negatives	No	Yes (after remove)	No
Memory Efficient	Yes	Less so	No



Thank you

- Author: Serhii Horbachov
- My LinkedIn: www.linkedin.com/in/serhiiorbachov
- Date: October 2025
- [Join Codeus community in Discord](#)
- [Join Codeus community in LinkedIn](#)

